

UNIVERSIDAD ADVENTISTA DEL PLATA
FACULTAD DE CIENCIAS ECONÓMICAS Y DE LA ADMINISTRACIÓN
ALGORITMOS Y ESTRUCTURA DE DATOS

LISTA DE EJERCICIOS N° 4 – Modularidad y Funciones (U4)

Sección 1: Introducción a funciones

1. Primera función*:

- a. Crear una función llamada `saludar()` que muestre por consola un mensaje de bienvenida.
- b. Luego modificarla para que reciba como parámetro el nombre de una persona:

```
saludar("Pedro");  
  
// por consola deberá mostrar  
// "Hola Pedro!"
```

2. Funciones con retorno*:

- a. Crear las siguientes funciones:

```
sumar(int a, int b)  
restar(int a, int b)  
multiplicar(int a, int b)
```

- b. Cada función deberá retornar el resultado correspondiente.
- c. Desde `main`, solicitar dos números al usuario y utilizar las funciones para mostrar los resultados.

3. Refactorización*:

- a. Tomar el ejercicio 15 de la Lista N°1, correspondiente al menú de operaciones. Refactorizarlo de manera que cada operación sea realizada mediante una función:

```
sumar(...)  
restar(...)  
multiplicar(...)  
dividir(...)
```

- b. El `main` deberá encargarse principalmente de la interacción con el usuario y de decidir qué operación realizar.

Sección 2: Parámetros y Retorno

4. Funciones sobre arrays*:

- a. Crear una función llamada `calcular suma`, que reciba un array de enteros y retorne la suma de sus elementos

```
calcularSuma(int[] numeros)
```

- b. Desde `main`:

- i. Crear un array
 - ii. Mostrar sus elementos
 - iii. Llamar a la función
 - iv. Mostrar el resultado
 - c. Luego, crear otra función llamada `calcularPromedio`, que reciba un array de enteros y devuelva el promedio de sus elementos. Probarla de la misma manera que la función `calcularSuma`.
- 5. Reutilizar funciones:
 - a. Tomar el ejercicio 7 de la Lista N° 3 que solicita obtener distintas estadísticas de un array. Refactorizarlo creando funciones para:
 - i. calcular suma;
 - ii. calcular promedio;
 - iii. obtener mayor;
 - iv. obtener menor;
 - v. contar positivos;
 - vi. contar negativos.
 - b. El programa principal deberá utilizar estas funciones para mostrar los resultados.

Sección 3: Strings + funciones

- 6. Procesamiento de Strings*:
 - a. Crear las siguientes funciones:

```
int contarVocales(String texto)
boolean esPalindromo(String texto)
```
 - b. Desde main, solicitar un texto al usuario y utilizar las funciones correspondientes
- 7. Función de búsqueda:
 - a. Crear una función que indique si un determinado carácter aparece dentro del texto:

```
boolean contiene(String texto, char caracter)
```
 - b. Luego crear otra función llamada `primeraPosición` que retorne la posición de la primera aparición del carácter:

```
int primeraPosicion(String texto, char caracter)
```
 - c. Se puede utilizar como base el ejercicio 4 de la Lista N°3

Sección 4: Scope y variables

8. Dónde existe esta variable*:

a. Analizar y corregir el siguiente código:

```
public static void main(String[] args) {  
    int numero = 10;  
  
    if (numero > 5) {  
        int resultado = numero * 2;  
        System.out.println(resultado);  
    }  
  
    System.out.println(resultado);  
}
```

- b. ¿Qué ocurre al intentar ejecutar el programa?
- c. ¿Cuál es el alcance de numero?
- d. ¿Cuál es el alcance de resultado?
- e. Modificar el código para que funcione correctamente.
- f. Después, crear una función que utilice una variable local y analizar desde dónde puede ser accedida.

Sección 5: Conversión y casting

9. Casting en funciones:

a. Crear una función que retorne el promedio de tres números enteros

```
double calcularPromedio(int a, int b, int c)
```

b. Luego probar las siguientes dos alternativas

```
return (double) (a + b + c) / 3;
```

```
return (double) ((a + b + c) / 3);
```

c. Explicar mediante una prueba de escritorio por qué se producen resultados diferentes.

Sección 6: Funciones matemáticas

10. Función de potencia*:

a. Crear una función que utilice Math.pow() para calcular la potencia.

```
double potencia(double base, double exponente)
```

Sección 7: Recursividad

11. Factorial recursivo*:

a. Crear una función que calcule el factorial de un número utilizando recursividad:

```
int factorial(int n)
```

- b. Por ejemplo, analizar:

```
factorial(5) → 120
```

```
5 × factorial(4)
4 × factorial(3)
3 × factorial(2)
2 × factorial(1)
1
```

- c. Identificar el caso base y el caso recursivo

12. Fibonacci recursivo:

- a. Tomar el ejercicio de Fibonacci de la Lista N°2 y modificarlo para implementarlo en una función recursiva que retorne el término correspondiente:

```
int fibonacci(int n)
```

Sección 8: Problema integrador

13. Refactorizando un programa:

- a. Tomar uno de los programas realizados anteriormente —por ejemplo, el menú de operaciones, el cálculo de KDA o el programa de estadísticas de un array— y refactorizarlo aplicando:
- funciones;
 - parámetros;
 - valores de retorno;
 - variables locales;
 - responsabilidad única.
- b. El objetivo no es modificar qué hace el programa, sino cómo está organizado.